

Scala

объектный и функциональный стиль

Вторая презентация: функции, классы, generics и организация кода

Команда 4. Бурачевский,
Киселева, Ятманова.
Группа 5030102/30202

Вторая презентация продолжает базу: теперь не отдельные конструкции, а способы строить программу

Было в 01

запуск в IntelliJ
val / var и типы
if / match
for / while / Iterator
List / Set / Map / Array
консоль и файлы

Сегодня в 02

функции
рекурсия и замыкания
class / object / trait
case class + pattern matching
generics
packages / import / export

Останется на 03

обработка ошибок
assert и отладка
конкурентность
JVM подробнее
управление памятью
практический итоговый пример

Функции: параметры и результат

03

Функция описывает входные данные, вычисление и тип результата

```
def greet(name: String, prefix: String = "Hello"): String =  
  s"$prefix, $name"
```

```
val a = greet("Anna")  
val b = greet(name = "Bob", prefix = "Hi")
```

Входные данные

параметры внутри круглых
скобок: name, prefix

Выходные данные

: String задает тип
возвращаемого значения

Default args

prefix имеет значение по
умолчанию, поэтому его можно
не передавать

Named args

аргументы можно передавать
по имени: name = "Bob"

Правило

return чаще всего не пишут: результатом функции становится последнее выражение в теле.

Рекурсия и хвостовая рекурсия

Рекурсивная функция вызывает сама себя; @tailrec помогает сделать это безопаснее

```
def factorial(n: Int): Int =  
  if n <= 1 then 1  
  else n * factorial(n - 1)
```

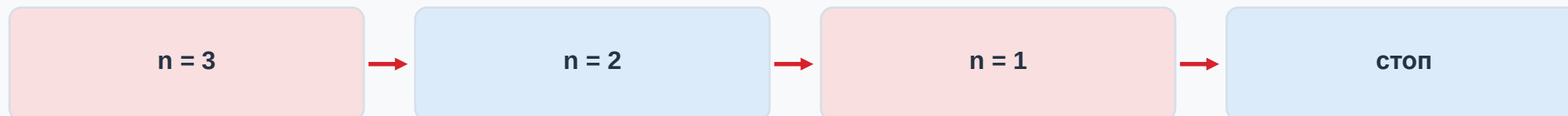
Обычная рекурсия

удобна для математических задач, деревьев и обходов, но может создавать цепочку вызовов.

```
import scala.annotation.tailrec  
  
@tailrec  
def sumTo(n: Int, acc: Int = 0): Int =  
  if n <= 0 then acc  
  else sumTo(n - 1, acc + n)
```

@tailrec

если рекурсия не хвостовая, компилятор сообщит об ошибке.



Функция может запомнить значения из окружающей области видимости

```
def makeAdder(delta: Int): Int => Int =  
  x => x + delta
```

```
val add10 = makeAdder(10)
```

```
println(add10(5)) // 15
```

```
var counter = 0
```

```
val next = () =>  
  counter += 1  
  counter
```

```
println(next()) // 1  
println(next()) // 2
```

Что захватывается

delta или counter из внешней области видимости

Где применяется

map/filter, фабрики функций, callbacks, настройки правил

Осторожно

замыкание на var может создавать скрытое изменяемое состояние

Функции высшего порядка

06

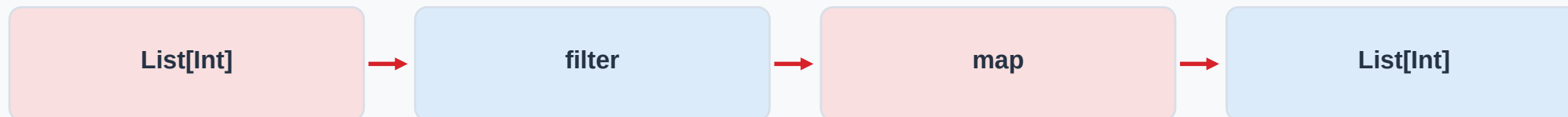
Функции можно передавать как обычные значения

```
def applyTwice(x: Int, f: Int => Int): Int =  
  f(f(x))
```

```
val result = applyTwice(3, _ + 1)  
// result = 5
```

```
val numbers = List(1, 2, 3, 4)
```

```
val result =  
  numbers  
    .filter(_ % 2 == 0)  
    .map(_ * 10)
```



Идея

Вместо того чтобы писать много похожих циклов, мы передаем поведение в виде функции.

Scala поддерживает обычное объектно-ориентированное программирование

```
class User(val name: String, var age: Int):
```

```
  def birthday(): Unit =  
    age += 1
```

```
  def info: String =  
    s"$name, $age"
```

```
val user = new User("Anna", 21)  
user.birthday()  
println(user.info)
```

Конструктор

name и age передаются при создании объекта.

Поля

val name - неизменяемое поле; var age - изменяемое.

Методы

birthday меняет состояние, info возвращает строку.

ООП-смысл

объект объединяет данные и операции над ними.

object и companion object

object - singleton; companion object хранит фабрики и “статические” методы

```
object MathUtils:  
  def square(x: Int): Int = x * x  
  
println(MathUtils.square(5))
```

```
class User(val name: String, val age: Int)  
  
object User:  
  def fromLine(line: String): User =  
    val parts = line.split(",")  
    new User(parts(0), parts(1).toInt)  
  
val user = User.fromLine("Anna,21")
```



class

описывает тип объектов и их поля/методы

object

единственный экземпляр; удобно для утилит, фабрик и точки входа

case class: удобная модель данных

09

Когда нужно описать данные, case class обычно лучше обычного class

```
case class Student(name: String, score: Int)
```

```
val s1 = Student("Anna", 95)  
val s2 = s1.copy(score = 100)
```

```
println(s1.name)  
println(s1 == Student("Anna", 95)) // true  
println(s2) // Student(Anna,100)
```

apply

можно писать Student(...)
без new

fields

параметры доступны как
поля

equals

сравнение по значениям

copy

создание измененной
копии

Pattern matching

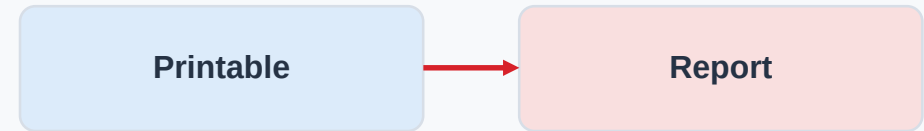
case class удобно разбирать в match по структуре объекта.

trait и mixin-композиция

10

trait задает общее поведение, которое можно подключать к классам

```
trait Printable:  
  def asText: String  
  
  def print(): Unit =  
    println(asText)  
  
class Report(val title: String) extends Printable:  
  def asText: String = s"Report: $title"  
  
val r = new Report("Scala")  
r.print()
```



Абстрактный метод

asText должен быть реализован классом.

Готовый метод

print уже реализован прямо в trait.

Использование

traits помогают собирать поведение из маленьких частей.

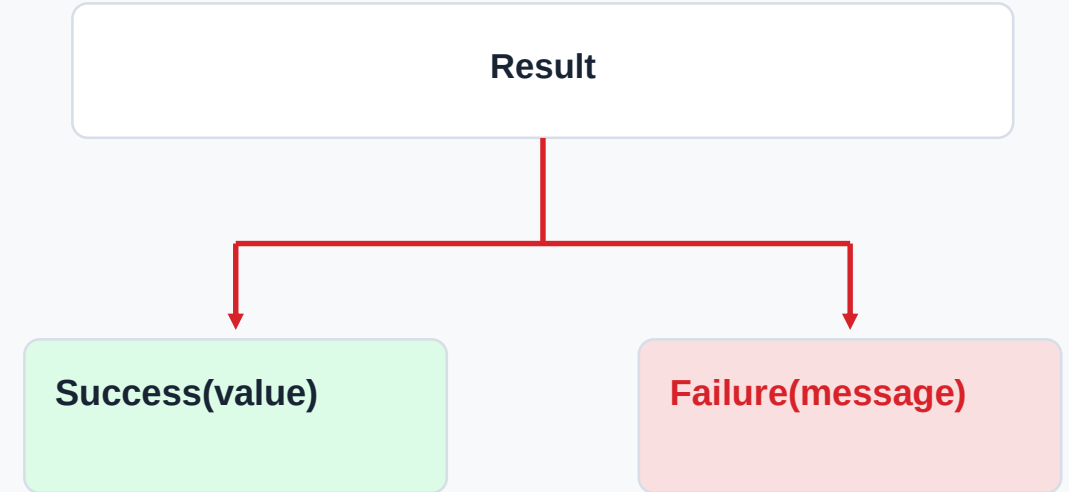
Pattern matching по объектам

11

case class + sealed trait превращают match в удобный разбор вариантов

```
sealed trait Result
case class Success(value: Int) extends Result
case class Failure(message: String) extends Result

def handle(result: Result): String =
  result match
    case Success(v) => s"OK: $v"
    case Failure(m) => s"Error: $m"
```



Зачем sealed

Если все варианты известны, компилятор может предупреждать о неполном match.

Generics: один код для разных типов данных

```
case class Box[A](value: A)

val intBox = Box(10)
val textBox = Box("Scala")

def first[A](items: List[A]): A =
  items.head

val n = first(List(1, 2, 3))
val s = first(List("a", "b"))
```

Декларация

Box[A] и first[A] объявляют параметр типа A.

Инстанцирование

Box(10) становится Box[Int], Box("Scala") - Box[String].

Использование

List[A] значит: список элементов некоторого типа A.

Польза

типобезопасность без копирования одного и того же кода.

Option[A]: generic на практике

13

Способ явно показать, что значения может не быть

```
def findUser(id: Int): Option[String] =  
  if id == 1 then Some("Anna")  
  else None  
  
val name =  
  findUser(1)  
    .map(_.toUpperCase)  
    .getOrElse("UNKNOWN")
```

Some(value)

значение есть

None

значения нет

map

применяется только если внутри Some; None остается None.

getOrElse

задает значение по умолчанию, если получился None.

Идея

Option заставляет обрабатывать отсутствие значения явно, а не надеяться, что null не придет.

Пакеты и import

14

Управление сложностью: раскладываем код по пространствам имен

```
package presentation.models

case class Student(name: String, score: Int)

package presentation.services

import presentation.models.Student
import java.time.LocalDate
import scala.collection.mutable.{Map => MutableMap}
```

```
02/
├── src/
│   ├── presentation/
│   │   ├── models/
│   │   │   └── Student.scala
│   │   └── services/
│   │       └── StudentService.scala
```

package

задает пространство имен

import

делает имя доступным в файле

alias

Map => MutableMap убирает конфликт имен

export в Scala 3

15

Экспортируем часть внутреннего API наружу

```
class StudentRepository:  
  def findAll(): List[Student] = ???  
  
class StudentService:  
  private val repo = new StudentRepository  
  
  export repo.findAll  
  
  def passed: List[Student] =  
    findAll().filter(_.score >= 60)
```



private repo

объект скрыт внутри сервиса

export repo.findAll

метод доступен как service.findAll()

Зачем

уменьшить связность и оставить чистый интерфейс модуля.

Модули и структура проекта

16

В маленьком проекте структура важнее сложной сборки

```
02/
├── README.md
├── src/
│   ├── 01_Functions.scala
│   ├── 02_Recursion.scala
│   └── presentation/
│       ├── models/
│       ├── repositories/
│       └── services/
└── FinalDemo.scala
```

models

структуры данных: case classes, enums

services

логика обработки и правила

repositories

источник данных: файл, список, БД

Main

точка входа @main и запуск демо

Правило

Файл должен отвечать за одну понятную часть программы. Это снижает сложность даже без сложной build-системы.

Коллекции глубже: immutable vs mutable

17

Scala поощряет неизменяемые коллекции, но не запрещает изменяемые

```
val xs = List(1, 2, 3)
val ys = xs :+ 4

println(xs) // List(1, 2, 3)
println(ys) // List(1, 2, 3, 4)
```

```
import scala.collection.mutable.ArrayBuffer

val buf = ArrayBuffer(1, 2, 3)
buf += 4

println(buf) // ArrayBuffer(1, 2, 3, 4)
```

```
val names = students
  .filter(_.score >= 60)
  .sortBy(_.name)
  .map(_.name)
```

Практическое правило

Сначала выбираем immutable. Mutable берем осознанно: для буфера, производительности или явного накопления.

В Scala нет Rust-like ownership: объекты живут на JVM и передаются как ссылки

```
val a = new StringBuilder("hi")  
val b = a  
  
b.append("!")  
  
println(a.toString) // hi!
```

```
case class Point(x: Int, y: Int)  
  
val p1 = Point(1, 2)  
val p2 = p1.copy(x = 10)  
  
println(p1) // Point(1,2)  
println(p2) // Point(10,2)
```

Ссылки

а и b могут указывать на один изменяемый объект.

GC

память освобождает JVM garbage collector.

Immutable-first

case class + copy помогают не менять исходный объект.

Вывод

Вместо ownership-контроля Scala часто опирается на неизменяемость и типы.

enum и extension methods - маленькие, но полезные возможности

```
enum Status:  
  case New, InProgress, Done  
  
def label(status: Status): String =  
  status match  
    case Status.New      => "new"  
    case Status.InProgress => "work"  
    case Status.Done      => "done"
```

```
extension (s: String)  
  def isBlankLike: Boolean =  
    s.trim.isEmpty  
  
println(" ".isBlankLike) // true
```

enum

тип с заранее известным набором вариантов.

extension

добавляет метод к существующему типу без наследования.

Итоговый пример: модель студентов

20

Одна программа объединяет case class, trait, generics, lambda и коллекции

```
case class Student(name: String, score: Int)

trait GradeRule:
  def passed(s: Student): Boolean

class MinScoreRule(limit: Int) extends GradeRule:
  def passed(s: Student): Boolean = s.score >= limit

class Report[A](val items: List[A]):
  def countWhere(p: A => Boolean): Int = items.count(p)

@main
def finalDemo(): Unit =
  val students = List(Student("Anna", 95), Student("Max", 48))
  val rule = new MinScoreRule(60)
  val report = new Report(students)
  println(report.countWhere(rule.passed))
```

case class

данные: Student(name, score)

trait + class

правило проверки: GradeRule / MinScoreRule

generic class

Report[A] работает с любым типом A

lambda / function value

countWhere принимает A => Boolean

collections

List + count/filter/map в реальном коде

Итоги второй презентации

21

Теперь видим, как из базовых конструкций складывается программа

- параметры, результат и default/named arguments
- рекурсия и @tailrec
- замыкания и функции высшего порядка
- class / object / companion object
- case class, trait и pattern matching по объектам
- generics: декларация, инстанцирование, использование
- packages / import / export / структура проекта
- ownership в Scala: ссылки, GC и immutable-first
- enum и extension methods

Что осталось на 03

ошибки, исключения, assert, конкурентность, JVM и память подробнее